

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS: FRANCISCO XAVIER REYES MUÑOZ

PARALELO: A

SEMESTRE: 5

ACTIVIDAD: #9 - PROYECTO PRÁCTICO - DESARROLLO DEL FRONTEND E INTEGRACIÓN FULL
STACK

PERIODO ABRIL 2026 - AGOSTO 2026



Actividades a desarrollar en la Unidad 4

Actividad # 9

Desarrollo e integración de un Sistema de Gestión de Incidentes Help Desk con MongoDB Atlas

Resumen

El presente informe documenta el desarrollo e integración de un Sistema de Gestión de Incidentes Help Desk con arquitectura cliente-servidor. El backend fue implementado con Node.js y Express, utiliza Mongoose como capa de modelado y se conecta a MongoDB Atlas para la persistencia de los tickets. El frontend fue construido con React y Vite e incorpora un panel de control con contadores, formulario de registro, listado de incidentes, edición y eliminación. La validación funcional se realizó mediante Postman, comprobando los métodos GET, POST, PUT y DELETE, además del endpoint de resumen utilizado por el dashboard. También se registran los problemas técnicos encontrados durante la implementación y las soluciones aplicadas.

Palabras clave: API REST, Help Desk, MongoDB Atlas, Mongoose, React, Vite, Postman.

Introducción

Los sistemas Help Desk permiten organizar, registrar y dar seguimiento a incidentes tecnológicos mediante tickets. Para esta actividad se desarrolló una aplicación web capaz de crear, consultar, actualizar y eliminar incidentes. La solución se estructuró en dos componentes: una API REST desarrollada con Node.js y Express y una interfaz React con Vite. Durante el desarrollo se utilizaron los puertos locales 5055 y 5173; para la entrega final, ambos componentes se publicaron mediante URL de Render y la persistencia se mantiene en MongoDB Atlas.



Express permite asociar métodos HTTP y rutas con funciones controladoras, lo que facilita la separación entre rutas, lógica de negocio y acceso a datos. Mongoose proporciona modelos, validaciones y operaciones de consulta para documentos almacenados en MongoDB. Por su parte, MongoDB Atlas requiere un usuario de base de datos y una dirección IP autorizada para permitir la conexión de una aplicación.

Objetivos

Objetivo general

Desarrollar e integrar un Sistema de Gestión de Incidentes Help Desk con operaciones CRUD, almacenamiento en MongoDB Atlas, interfaz React y validación de los endpoints mediante Postman.

Objetivos específicos

- Configurar una API REST con Node.js y Express, usando el puerto 5055 para desarrollo local y una URL pública de Render para producción.
- Conectar el backend con MongoDB Atlas mediante Mongoose y variables de entorno.
- Definir un modelo de ticket con validaciones para título, descripción, categoría, prioridad y estado.
- Implementar endpoints para crear, listar, buscar, actualizar y eliminar tickets.
- Crear un endpoint de resumen para alimentar los contadores del dashboard.
- Desarrollar una interfaz React con formulario, listado, filtros, edición, eliminación y diseño responsivo.
- Comprobar el funcionamiento del sistema con Postman, MongoDB Atlas y el navegador utilizando las URL públicas de producción.



Fundamentación tecnológica

Tabla 1

Tecnologías utilizadas

Tecnología	Aplicación en el proyecto
Node.js	Ejecución del backend y lectura de variables mediante process.env.
Express	Rutas HTTP, middleware, recepción de JSON y respuestas.
MongoDB Atlas	Base de datos en la nube para la colección tickets.
Mongoose	Esquema, validaciones y operaciones CRUD.
React	Interfaz basada en componentes y estado.
Vite	Servidor de desarrollo y configuración del frontend.
Postman	Pruebas manuales de la API REST.

Node.js reconoce archivos de variables de entorno como conjuntos de pares clave-valor que pueden consultarse desde process.env (Node.js, s. f.). React utiliza createRoot para montar la aplicación en un nodo del documento HTML (React, s. f.), mientras que Vite integra el complemento oficial @vitejs/plugin-react desde vite.config.js (Vite, s. f.).

Metodología de desarrollo

Se aplicó un proceso incremental. Primero se configuró el backend y la conexión con MongoDB Atlas; después se definió el modelo Ticket y se implementaron las rutas CRUD. Una vez comprobada la API en Postman, se desarrolló el frontend React y se conectaron los componentes con el backend mediante fetch. Finalmente se realizaron pruebas integrales, se corrigieron errores de configuración y se prepararon las evidencias.



Arquitectura y estructura del proyecto

```
C:\HelpDesk-Backend\  
├── backend\  
│   ├── src\  
│   │   ├── config\database.js  
│   │   ├── controllers\ticketController.js  
│   │   ├── models\Ticket.js  
│   │   ├── routes\ticketRoutes.js  
│   │   ├── app.js  
│   │   └── server.js  
│   ├── .env  
│   ├── .gitignore  
│   └── package.json  
└── frontend\  
    ├── public\  
    ├── src\  
    │   ├── components\  
    │   │   ├── Dashboard.jsx  
    │   │   ├── ListaTickets.jsx  
    │   │   ├── Navbar.jsx  
    │   │   └── RegistrarIncidente.jsx  
    │   ├── services\api.js  
    │   ├── App.jsx  
    │   ├── main.jsx  
    │   └── style.css  
    ├── .env  
    ├── index.html  
    ├── package.json  
    └── vite.config.js
```

En producción, el usuario accede al frontend mediante <https://helpdesk-frontend-react.onrender.com>. El frontend consume la API pública alojada en <https://helpdesk-backend-ymmq.onrender.com>; el backend procesa las solicitudes REST y persiste la información en MongoDB Atlas. Las direcciones localhost se conservan únicamente para desarrollo local y no forman parte de la arquitectura final de entrega.



Implementación del backend

package.json

Este archivo define los comandos de ejecución y las dependencias del backend. El comando `npm run dev` utiliza `nodemon` para reiniciar el servidor durante el desarrollo.

```
{
  "name": "helpdesk-backend",
  "version": "1.0.0",
  "main": "src/server.js",
  "scripts": {
    "start": "node src/server.js",
    "dev": "nodemon src/server.js"
  },
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.6.1",
    "express": "^5.1.0",
    "mongoose": "^9.8.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.14"
  }
}
```

Variables de entorno

El archivo `backend/.env` almacena el puerto, la URI de conexión y el origen permitido.

La URI real debe permanecer oculta.

```
PORT=5055 # solo desarrollo local; Render proporciona process.env.PORT en producción
MONGODB_URI=[cadena privada de MongoDB Atlas configurada como variable de entorno en
Render]
FRONTEND_URL=https://helpdesk-frontend-react.onrender.com
```

Conexión database.js

La función valida que `MONGODB_URI` exista y tenga un prefijo válido antes de llamar a `mongoose.connect()`. Esto evita el error producido cuando Mongoose recibe `undefined`.

```
const mongoose = require("mongoose");

const conectarBaseDatos = async () => {
  const uri = process.env.MONGODB_URI;

  if (!uri || typeof uri !== "string" || uri.trim() === "") {
    throw new Error("MONGODB_URI no está definida. Revisa backend/.env.");
  }

  if (!uri.startsWith("mongodb://") && !uri.startsWith("mongodb+srv://")) {
    throw new Error("MONGODB_URI no tiene un formato válido.");
  }
}
```



```
}

const conexion = await mongoose.connect(uri.trim());
console.log("MongoDB Atlas conectado correctamente");
return conexion;
};

module.exports = conectarBaseDatos;
```

Modelo Ticket.js

El esquema define los campos obligatorios y las opciones permitidas. Mongoose ejecuta validaciones al guardar documentos y también puede aplicarlas durante actualizaciones cuando se activa `runValidators`.

```
const ticketSchema = new mongoose.Schema(
  {
    titulo: { type: String, required: true, trim: true, minlength: 3, maxlength: 120 },
    descripcion: { type: String, required: true, trim: true, minlength: 5, maxlength: 2000 },
    categoria: {
      type: String,
      enum: ["Hardware", "Software", "Red", "Accesos", "Impresoras", "Otro"],
      default: "Otro"
    },
    prioridad: {
      type: String,
      enum: ["Baja", "Media", "Alta", "Crítica"],
      default: "Media"
    },
    estado: {
      type: String,
      enum: ["Abierto", "En proceso", "Cerrado"],
      default: "Abierto"
    }
  },
  { timestamps: true, versionKey: false }
);
```

Controlador ticketController.js

El controlador contiene la lógica de negocio. `obtenerTickets` lista los documentos; `crearTicket` guarda un registro; `actualizarTicket` utiliza `findByIdAndUpdate`; `eliminarTicket` utiliza `findByIdAndDelete`; y `obtenerResumenTickets` calcula los indicadores con `countDocuments`.

```
const obtenerResumenTickets = async (req, res) => {
  const [total, abiertos, enProceso, cerrados, prioridadAlta, criticos] =
    await Promise.all([
      Ticket.countDocuments(),
      Ticket.countDocuments({ estado: "Abierto" }),
```



```
Ticket.countDocuments({ estado: "En proceso" }},
Ticket.countDocuments({ estado: "Cerrado" }},
Ticket.countDocuments({ prioridad: "Alta" }},
Ticket.countDocuments({ prioridad: "Crítica" })
]);

return res.status(200).json({
  exito: true,
  resumen: { total, abiertos, enProceso, cerrados, prioridadAlta, criticos }
});
};

const actualizarTicket = async (req, res) => {
  const ticketActualizado = await Ticket.findByIdAndUpdate(
    req.params.id,
    req.body,
    { new: true, runValidators: true }
  );

  if (!ticketActualizado) {
    return res.status(404).json({ exito: false, mensaje: "Ticket no encontrado" });
  }

  return res.status(200).json({
    exito: true,
    mensaje: "Ticket actualizado correctamente",
    ticket: ticketActualizado
  });
};

const eliminarTicket = async (req, res) => {
  const ticketEliminado = await Ticket.findByIdAndDelete(req.params.id);

  if (!ticketEliminado) {
    return res.status(404).json({ exito: false, mensaje: "Ticket no encontrado" });
  }

  return res.status(200).json({
    exito: true,
    mensaje: "Ticket eliminado correctamente",
    ticketEliminado
  });
};
```

Rutas ticketRoutes.js

La ruta /resumen se declara antes de /:id para evitar que Express interprete resumen como un identificador.

```
router.get("/resumen", obtenerResumenTickets);
router.get("/", obtenerTickets);
router.get("/:id", obtenerTicketPorId);
router.post("/", crearTicket);
router.put("/:id", actualizarTicket);
router.delete("/:id", eliminarTicket);
```



app.js y server.js

app.js configura CORS, JSON, rutas y errores. En producción, FRONTEND_URL autoriza el origen público del frontend. server.js conecta MongoDB Atlas y utiliza process.env.PORT, con 5055 únicamente como valor alternativo para desarrollo local. El servidor se enlaza a 0.0.0.0 para recibir tráfico público en Render.

```
app.use(cors({
  origin: process.env.FRONTEND_URL || "http://localhost:5173", // fallback local
  methods: ["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
  allowedHeaders: ["Content-Type", "Authorization"]
}));

app.use(express.json({ limit: "1mb" }));
app.use("/tickets", ticketRoutes);
```

```
const PORT = Number(process.env.PORT) || 5055;

const iniciarServidor = async () => {
  await conectarBaseDatos();

  app.listen(PORT, "0.0.0.0", () => {
    console.log(`Servidor activo en el puerto ${PORT}`);
    // Producción: https://helpdesk-backend-ymmq.onrender.com/tickets
    // Producción: https://helpdesk-backend-ymmq.onrender.com/tickets/resumen
  });
};

iniciarServidor();
```

Tabla 2

Endpoints implementados

Método	Ruta	Finalidad
GET	/	Verificar que la API esté activa.
GET	/tickets	Listar tickets.
GET	/tickets/resumen	Obtener contadores.
GET	/tickets/:id	Buscar por ObjectId.
POST	/tickets	Crear ticket.
PUT	/tickets/:id	Editar ticket.
DELETE	/tickets/:id	Eliminar ticket.



Implementación del frontend

vite.config.js

El archivo se ubica en la raíz de frontend y configura el complemento oficial de React y el puerto 5173.

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    strictPort: true
  }
});
```

main.jsx

main.jsx crea la raíz de React e importa el archivo real style.css. La importación styles.css causaba un error porque ese archivo no existía.

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./style.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

services/api.js

Este servicio centraliza fetch y utiliza VITE_API_URL para consumir el backend. En el Static Site de Render, VITE_API_URL se configura con <https://helpdesk-backend-yymmq.onrender.com>, evitando cualquier dependencia de localhost en producción.

```
const API_BASE_URL = (
  import.meta.env.VITE_API_URL || "http://localhost:5055"
).replace(/\/+$/, "");

const API_URL = `${API_BASE_URL}/tickets`;

const procesarRespuesta = async (respuesta) => {
  const datos = await respuesta.json().catch(() => ({}));
  if (!respuesta.ok) {
    throw new Error(datos.mensaje || "Ocurrió un error en la solicitud");
  }
};
```



```
    }  
    return datos;  
  };  
  
  export const obtenerTickets = async () => {  
    const respuesta = await fetch(API_URL);  
    const datos = await procesarRespuesta(respuesta);  
    return datos.tickets || [];  
  };  
  
  export const obtenerResumen = async () => {  
    const respuesta = await fetch(`${API_URL}/resumen`);  
    const datos = await procesarRespuesta(respuesta);  
    return datos.resumen;  
  };  
  
  export const crearTicket = async (ticket) =>  
    procesarRespuesta(await fetch(API_URL, {  
      method: "POST",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify(ticket)  
    }));  
  
  export const actualizarTicket = async (id, ticket) =>  
    procesarRespuesta(await fetch(`${API_URL}/${id}`, {  
      method: "PUT",  
      headers: { "Content-Type": "application/json" },  
      body: JSON.stringify(ticket)  
    }));  
  
  export const eliminarTicket = async (id) =>  
    procesarRespuesta(await fetch(`${API_URL}/${id}`, { method: "DELETE" }));
```

App.jsx

App.jsx administra las vistas, los tickets, los contadores, el ticket en edición y los mensajes. Después de cada operación vuelve a cargar la lista y el resumen.

```
const cargarDatos = useCallback(async () => {  
  setCargando(true);  
  
  try {  
    const [listaTickets, datosResumen] = await Promise.all([  
      obtenerTickets(),  
      obtenerResumen()  
    ]);  
  
    setTickets(listaTickets);  
    setResumen(datosResumen || resumenInicial);  
  } finally {  
    setCargando(false);  
  }  
}, []);
```

Dashboard.jsx

El dashboard muestra seis indicadores con clases CSS únicas para diferenciar visualmente cada estado.



```
const tarjetasDashboard = [
  { campo: "total", titulo: "Total de tickets", clase: "tarjeta-marino" },
  { campo: "abiertos", titulo: "Tickets abiertos", clase: "tarjeta-azul" },
  { campo: "enProceso", titulo: "En proceso", clase: "tarjeta-turquesa" },
  { campo: "cerrados", titulo: "Tickets cerrados", clase: "tarjeta-verde" },
  { campo: "prioridadAlta", titulo: "Prioridad alta", clase: "tarjeta-naranja" },
  { campo: "criticos", titulo: "Tickets críticos", clase: "tarjeta-roja" }
];
```

ListaTickets.jsx

El componente muestra la tabla, búsqueda y filtro, abre la edición y solicita confirmación antes de eliminar.

```
<button className="boton-accion editar" onClick={() => editarTicket(ticket)}>
  Editar
</button>

<button className="boton-accion eliminar" onClick={() =>
confirmarEliminacion(ticket)}>
  Eliminar
</button>
```

RegistrarIncidente.jsx

El formulario sirve para crear y editar. Cuando existe ticketEditando, los campos se cargan mediante useEffect y el botón cambia a Guardar cambios.

Diseño visual

Se aplicó una paleta sobria con azul marino, azul, turquesa, verde, naranja y rojo. Cada contador posee un color único y la interfaz incluye adaptación para pantallas medianas y móviles.

Configuración de MongoDB Atlas

MongoDB Atlas diferencia el usuario de la plataforma del usuario de base de datos. Para conectar la aplicación se requiere un usuario de base de datos y una dirección autorizada en la lista de acceso IP.

1. Ingresar a MongoDB Atlas y crear o seleccionar un proyecto.



2. Crear un clúster y esperar a que se encuentre disponible.
3. Abrir Security > Database Access y crear un usuario de base de datos.
4. Abrir Security > Network Access y agregar la dirección IP actual.
5. En Database > Clusters seleccionar Connect y luego Drivers.
6. Elegir Node.js y copiar la cadena mongodb+srv://.
7. Reemplazar <password> por la contraseña del usuario de base de datos.
8. Guardar la cadena únicamente en backend/.env como MONGODB_URI.
9. Ejecutar npm run dev y comprobar la conexión.
10. Ingresar a Data Explorer y verificar Cluster0 > HelpDesk > tickets.

Comandos de ejecución local (solo desarrollo)

```
# Terminal 1
cd C:\HelpDesk-Backend\backend
npm install
npm run dev

# Terminal 2
cd C:\HelpDesk-Backend\frontend
npm install
npm run dev
```



Comprobación paso a paso en Postman

Postman permite seleccionar el método, escribir la URL, agregar encabezados o cuerpo y enviar la solicitud. Para POST y PUT se utiliza Body > raw > JSON.

Prueba 1. Verificar la API

Método: GET. URL: <https://helpdesk-backend-ymmq.onrender.com/>. Comprobar código 200 y el mensaje de funcionamiento del servicio público.

Prueba 2. Crear un ticket

Método: POST. URL: <https://helpdesk-backend-ymmq.onrender.com/tickets>. Elegir Body > raw > JSON y comprobar código 201.

```
{
  "titulo": "Servidor principal no responde",
  "descripcion": "El servidor perdió conexión y los servicios internos no están disponibles.",
  "categoria": "Red",
  "prioridad": "Crítica",
  "estado": "Abierto"
}
```

Prueba 3. Listar tickets

Método: GET. URL: <https://helpdesk-api-4lhc.onrender.com/tickets/resumen> Verificar cantidad y arreglo tickets.

Prueba 4. Buscar por ID

Método: GET. URL: <https://helpdesk-api-4lhc.onrender.com/tickets/6a77be38d31a1e2e87951ff1> Reemplazar ID por el ObjectId real del ticket.

Prueba 5. Consultar resumen

Método: GET. URL: <https://helpdesk-api-4lhc.onrender.com/tickets/resumen> Verificar los seis contadores.



Prueba 6. Editar un ticket

Método: PUT. URL: <https://helpdesk-api-4lhc.onrender.com/tickets/6a77bc9483f987b47ac14091> Enviar campos modificados y comprobar código 200.

```
{
  "estado": "En proceso",
  "prioridad": "Alta"
}
```

Prueba 7. Eliminar un ticket

Método: DELETE. URL: <https://helpdesk-api-4lhc.onrender.com/tickets/6a77bc9483f987b47ac14091> Comprobar el mensaje de eliminación y volver a listar.

Respuesta esperada del resumen

```
{
  "exito": true,
  "resumen": {
    "total": 1,
    "abiertos": 1,
    "enProceso": 0,
    "cerrados": 0,
    "prioridadAlta": 0,
    "criticos": 1
  }
}
```

Validación de errores

Se recomienda probar un identificador inválido, un ObjectId que no exista y un cuerpo sin título o descripción. La API debe responder con códigos 400 o 404 y mensajes controlados.



Problemas encontrados y soluciones

Tabla 4

Incidencias técnicas corregidas

Problema	Causa	Solución
MONGODB_URI undefined	El .env no estaba en backend o se cargaba tarde.	Se cargó .env desde server.js y se validó la variable.
vite.config.js no resolvía	El archivo no existía o estaba dañado.	Se creó en la raíz de frontend con sintaxis ESM correcta.
Unterminated string literal	El archivo contenía '@' o comillas externas.	Se reemplazó todo el contenido sin comillas alrededor de las líneas.
Expected ')'	Las líneas import estaban guardadas como cadenas literales.	Se dejó código JavaScript normal.
No se resolvía styles.css	El archivo real se llamaba style.css.	Se corrigió import "./style.css".
Frontend sin comunicación	La URL no coincidía con el backend.	Se configuró VITE_API_URL=https://helpdesk-backend-ymmj.onrender.com en el Static Site de Render y api.js construye la ruta /tickets.
CORS	El origen permitido no era el de Vite.	Se configuró FRONTEND_URL=https://helpdesk-frontend-react.onrender.com en el backend desplegado en Render.

Pruebas del frontend

1. Abrir <https://helpdesk-frontend-react.onrender.com> en Chrome, preferiblemente en una ventana de incógnito y con las terminales locales cerradas.
2. Comprobar las seis tarjetas del dashboard.
3. Registrar un ticket desde el formulario.



4. Verificar que aparezca en la tabla y se actualicen los contadores.
5. Editar el estado o la prioridad y guardar.
6. Probar la búsqueda y el filtro por estado.
7. Eliminar con confirmación.
8. Revisar MongoDB Atlas (Cluster0 > HelpDesk > tickets) para confirmar que los cambios realizados desde la URL pública quedaron persistidos.

Control de versiones y repositorios

```
# Backend: rama develop
git add .
git commit -m "Configurar backend para frontend público"
git push -u origin develop
```

Despliegue en la nube

La solución final se configuró para operar mediante servicios públicos. MongoDB Atlas proporciona la persistencia remota; Render aloja el backend como Web Service y el frontend como Static Site. En producción, el frontend consume exclusivamente la URL pública del backend mediante VITE_API_URL, mientras que el backend autoriza el origen público del cliente mediante FRONTEND_URL. Las direcciones localhost quedan limitadas a pruebas de desarrollo.

Arquitectura final de producción

```
MongoDB Atlas (Cluster0 > HelpDesk > tickets)
    ↑ MONGODB_URI
Backend público: https://helpdesk-backend-ymmq.onrender.com
    ↑ ↓ API REST
Frontend público: https://helpdesk-frontend-react.onrender.com
```

Matriz de cumplimiento del despliegue

Requisito	Implementación	Dirección / configuración	Estado
Persistencia	MongoDB Atlas	Cluster0 > HelpDesk; conexión privada mediante MONGODB_URI	Configurado



Servidor	Render Web Service	https://helpdesk-backend-ymmq.onrender.com	Live
Cliente	Render Static Site (React + Vite)	https://helpdesk-frontend-react.onrender.com	Publicado
Producción	Sin dependencia de localhost	VITE_API_URL → backend público; FRONTEND_URL → frontend público	Verificar CRUD

Backend en Render

El backend se encuentra publicado como Web Service en <https://helpdesk-backend-ymmq.onrender.com>. El repositorio utilizado es <https://github.com/freyes7194-svg/HelpDesk-Backend.git>, con la rama develop para los cambios de despliegue. El comando de inicio es `npm start`, que ejecuta `node src/server.js`. En producción no se define manualmente `PORT=5055`: el código prioriza `process.env.PORT` y conserva 5055 únicamente como fallback local. Render requiere que la aplicación escuche en 0.0.0.0 para recibir tráfico público.

Variables del backend en Render:

- MONGODB_URI = cadena privada de conexión a MongoDB Atlas (no debe mostrarse en capturas).
- NODE_ENV = production.
- FRONTEND_URL = <https://helpdesk-frontend-react-zamu.onrender.com>

Frontend en Render

El frontend React + Vite se publica como Static Site en <https://helpdesk-frontend-react.onrender.com>. El repositorio es <https://github.com/freyes7194-svg/helpdesk-frontend-react.git>, rama main. El proceso de construcción utiliza `npm install && npm run build` y el directorio publicado es dist. La variable VITE_API_URL debe contener <https://helpdesk-backend-ymmq.onrender.com>, sin agregar /tickets al final si api.js construye esa ruta internamente.

Configuración del Static Site:

- Tipo de servicio: Static Site.
- Branch: main.
- Build Command: `npm install && npm run build`.
- Publish Directory: dist.
- VITE_API_URL = <https://helpdesk-api-4lhc.onrender.com>



Verificación de funcionamiento 100 % web

Para demostrar que la aplicación no depende de localhost se debe realizar la prueba final con las terminales locales cerradas. La evidencia debe mostrar el acceso al frontend público, la creación de un ticket, su edición, el cambio de estado con actualización de los contadores, la eliminación del registro y la persistencia correspondiente en MongoDB Atlas. Esta prueba es la evidencia principal del requisito de funcionamiento completamente web.

1. Abrir <https://helpdesk-frontend-react.onrender.com> en una ventana de incógnito.
2. Crear un ticket desde el formulario público.
3. Comprobar que el ticket aparece en el listado y cambia el contador del dashboard.
4. Editar el estado o la prioridad y guardar.
5. Eliminar el ticket mediante la confirmación de la interfaz.
6. Actualizar la página con Ctrl + F5 y verificar que los cambios permanecen.
7. Abrir MongoDB Atlas y confirmar los datos en Cluster0 > HelpDesk > tickets.
8. Repetir GET, POST, PUT y DELETE en Postman usando <https://helpdesk-backend-ymmq.onrender.com>, no localhost.

Repositorios de la solución

Backend: <https://github.com/freyes7194-svg/HelpDesk-Backend.git> (rama de despliegue: develop).

Frontend: <https://github.com/freyes7194-svg/helpdesk-frontend-react.git> (rama de despliegue: main).

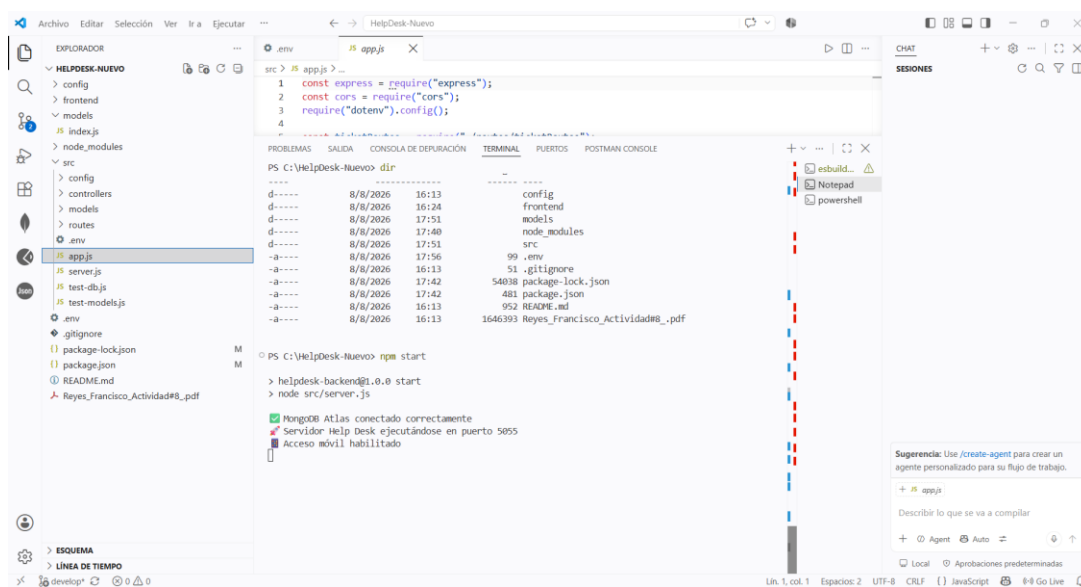
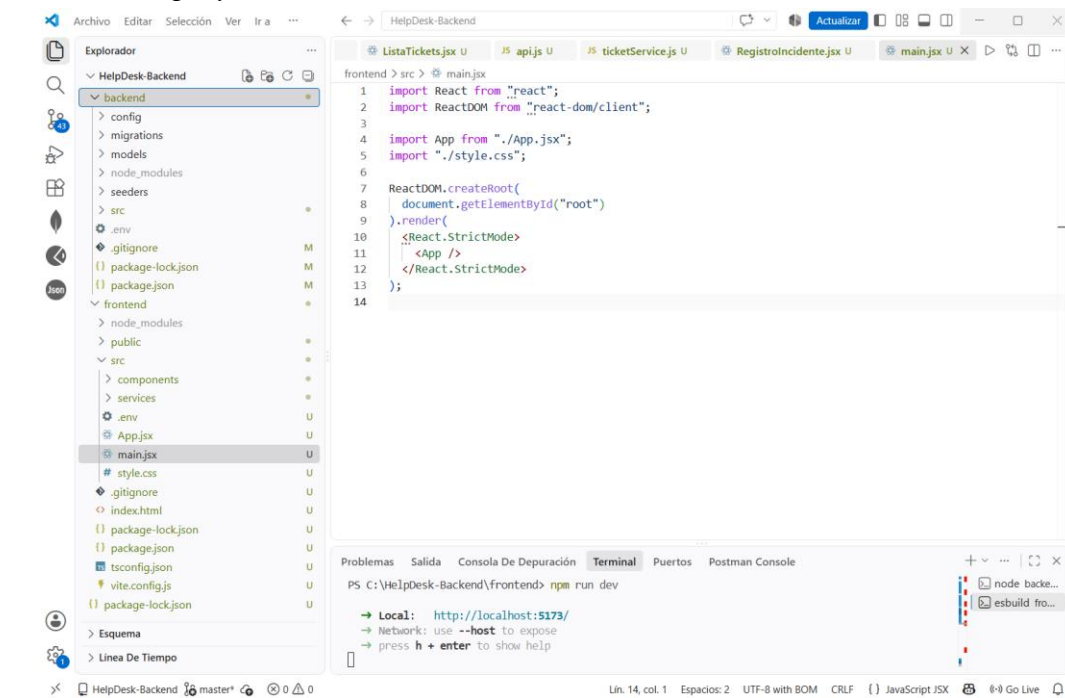
Evidencias

En los espacios siguientes deben insertarse capturas propias, claras y legibles. La contraseña y la URI completa de MongoDB Atlas no deben aparecer.



Figura 1

Estructura del proyecto en Visual Studio Code

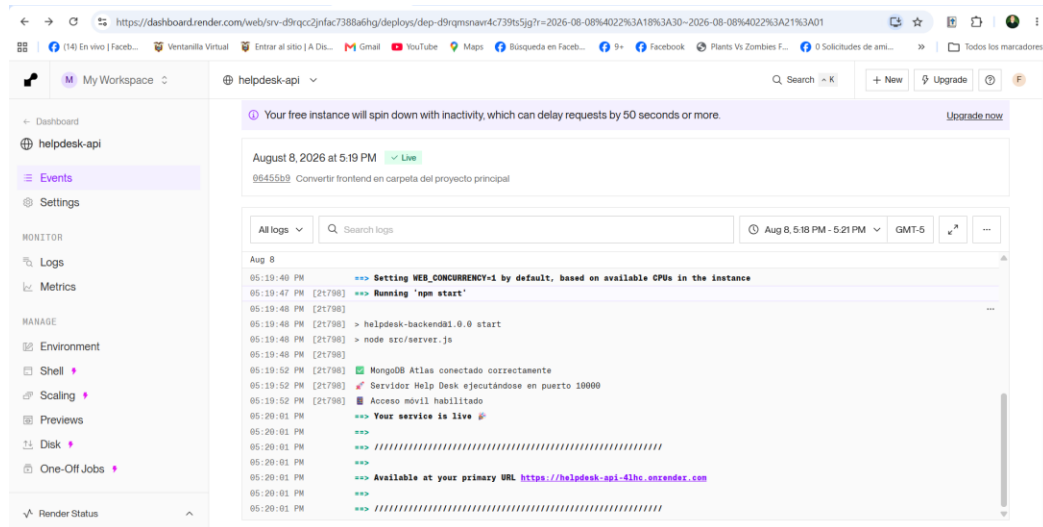


Nota. Mostrar backend, frontend y archivos principales.



Figura 2

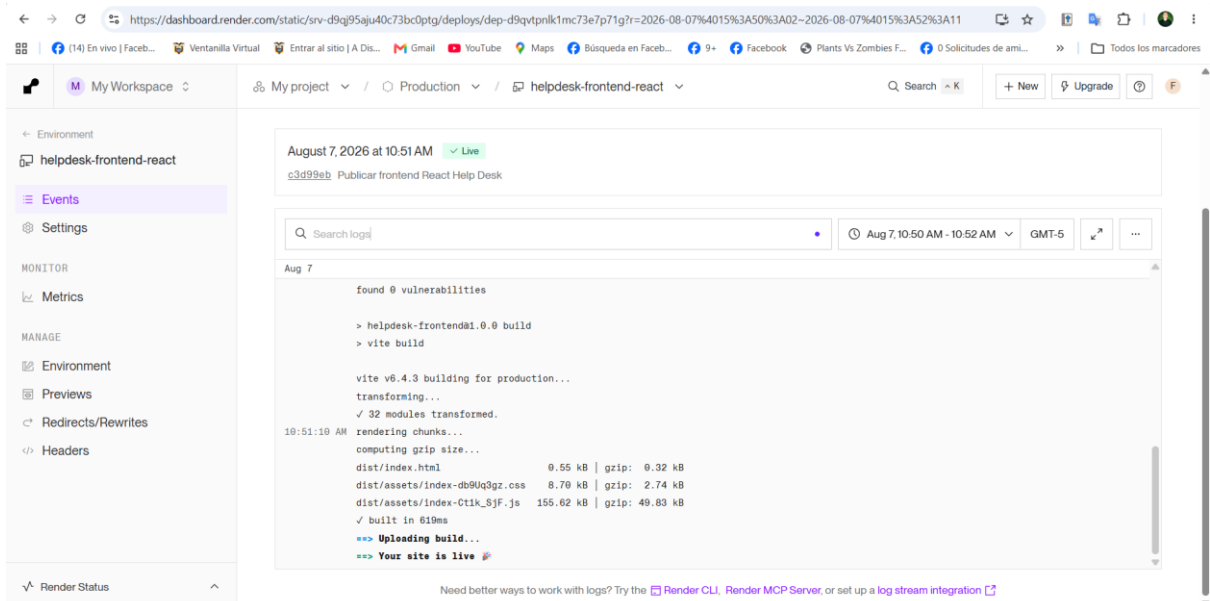
Backend desplegado y activo en Render



Nota. Render confirma “Your service is live” y muestra la URL pública <https://helpdesk-backend-ymmq.onrender.com>.

Figura 3

Frontend publicado como Static Site en Render

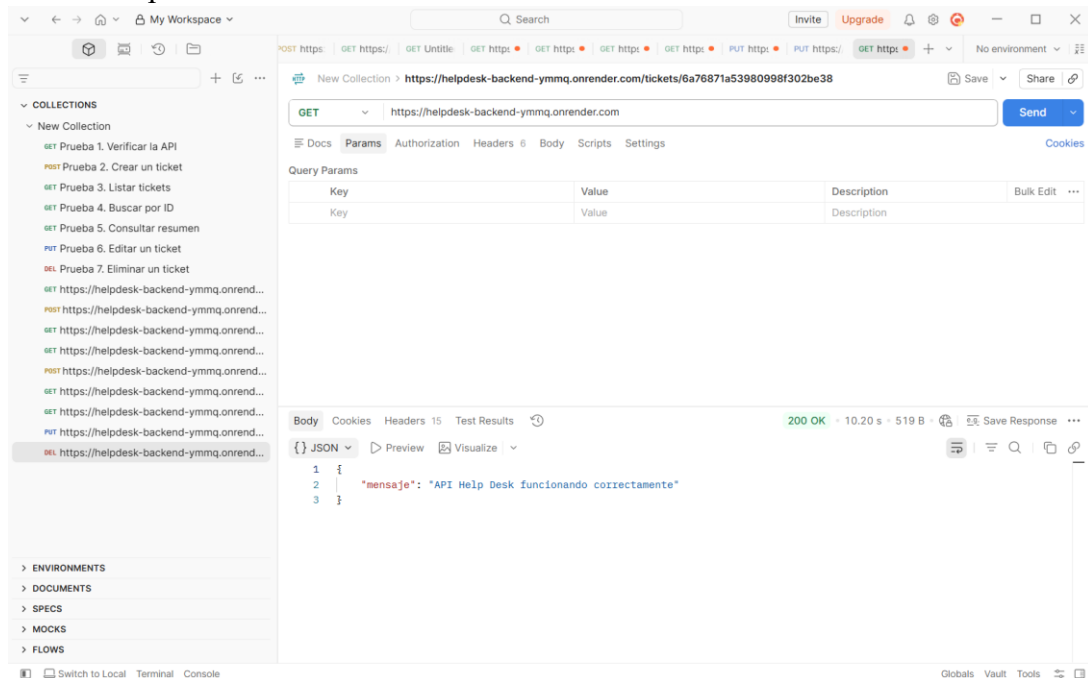




Nota. Insertar captura final de <https://helpdesk-frontend-react.onrender.com> mostrando la interfaz cargada desde la web.

Figura 4

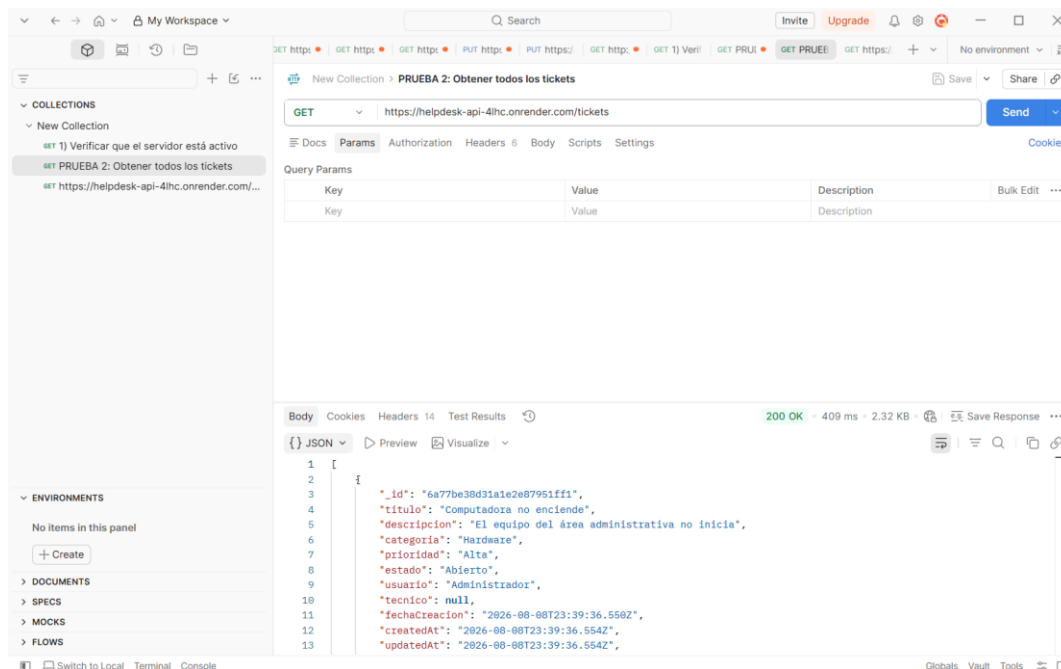
GET de la API pública en Postman



Nota. Mostrar GET <https://helpdesk-api-4lhc.onrender.com/>, código 200 y respuesta.

Figura 5

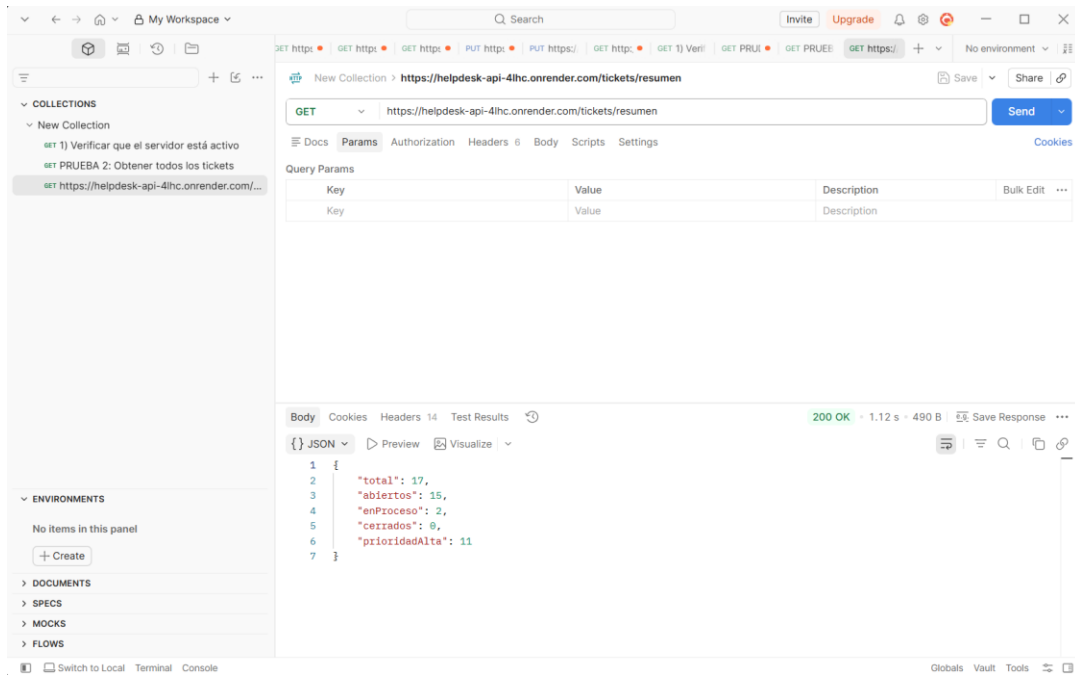
GET PRUEBA 2: Obtener todos los tickets



Nota. Mostrar GET <https://helpdesk-api-4lhc.onrender.com/tickets>, cuerpo JSON y respuesta 201.

Figura 6

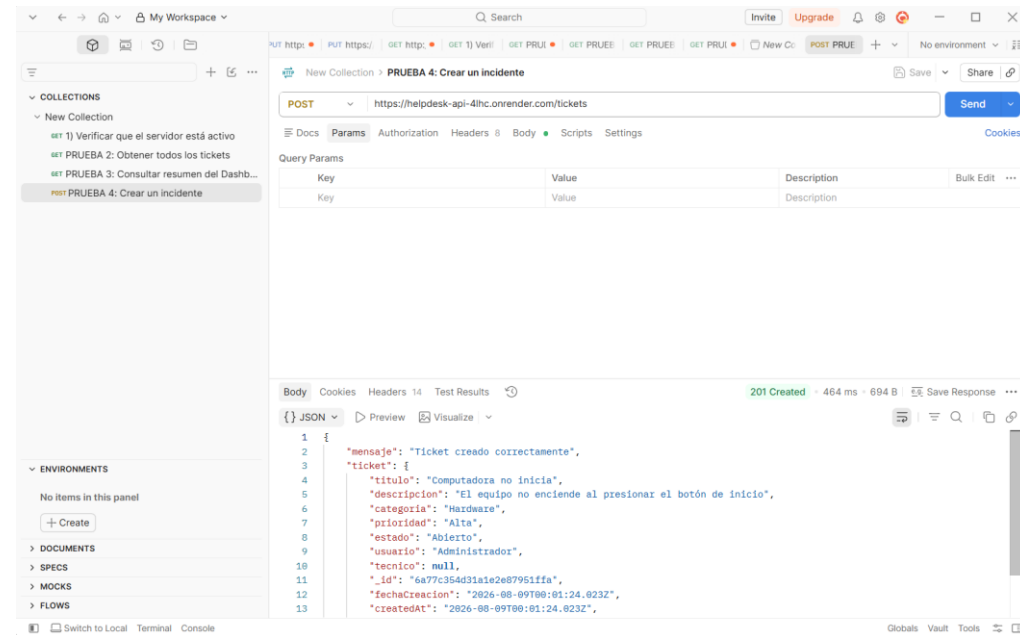
GET PRUEBA 3: Consultar resumen del Dashboard



Nota. Mostrar GET <https://helpdesk-api-4lhc.onrender.com/tickets/resumen>, cantidad y arreglo tickets.

Figura 7

PRUEBA 4: Crear un incidente

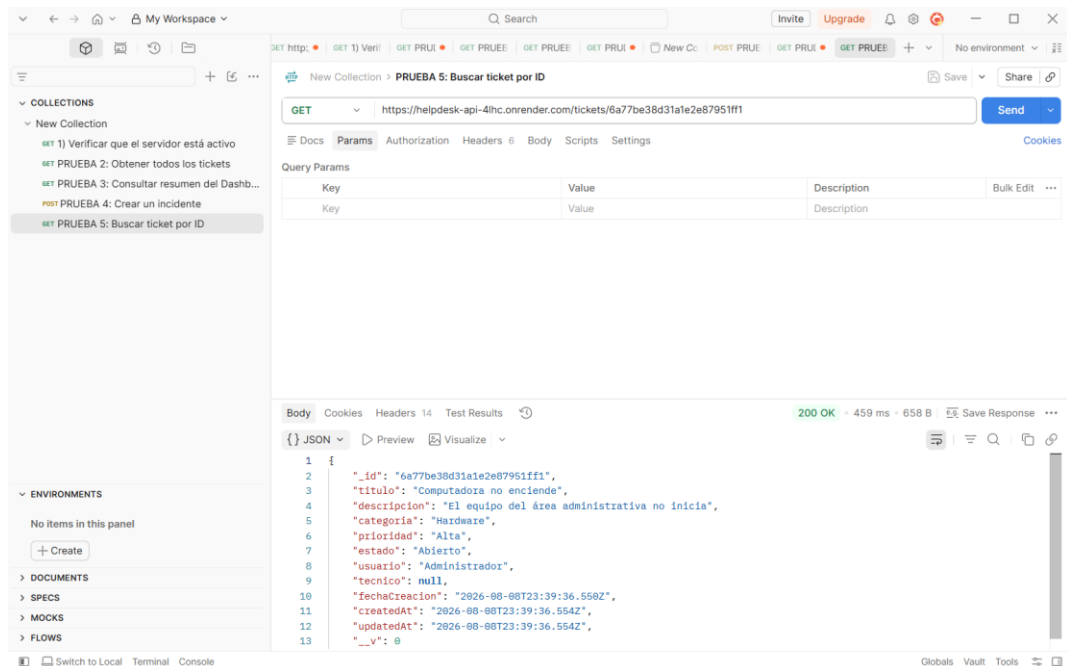


Nota. Mostrar GET <https://helpdesk-api-4lhc.onrender.com/tickets> resumen y los seis contadores.



Figura 8

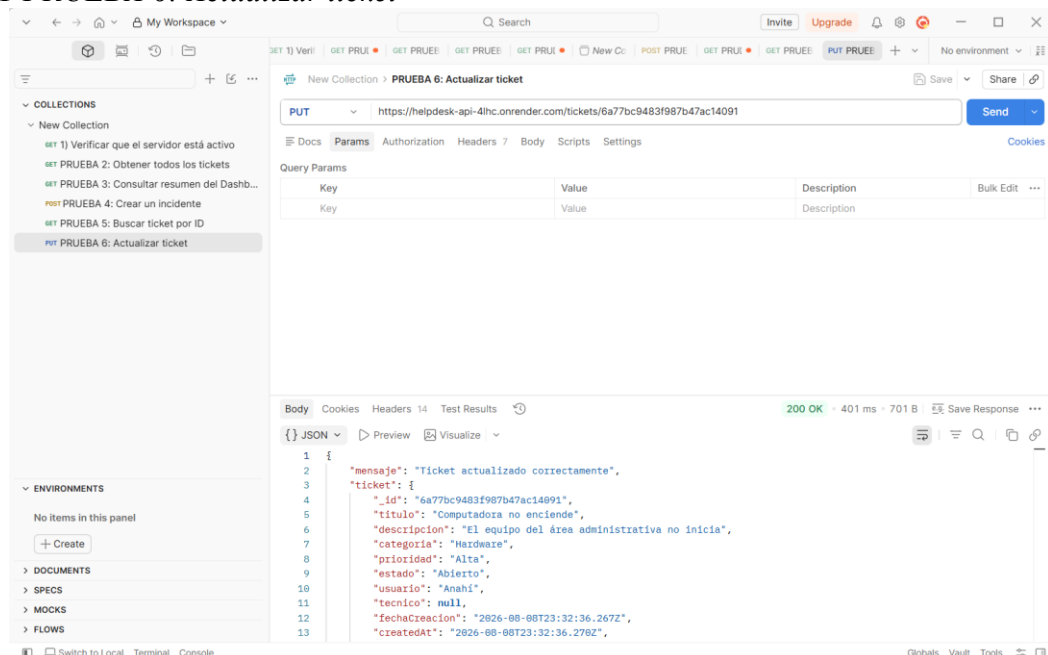
GET de un ticket por ID



Nota. Mostrar GET <https://helpdesk-api-4lhc.onrender.com/tickets/6a77be38d31a1e2e87951ff1> con el ObjectId real.

Figura 9

PUT PRUEBA 6: Actualizar ticket

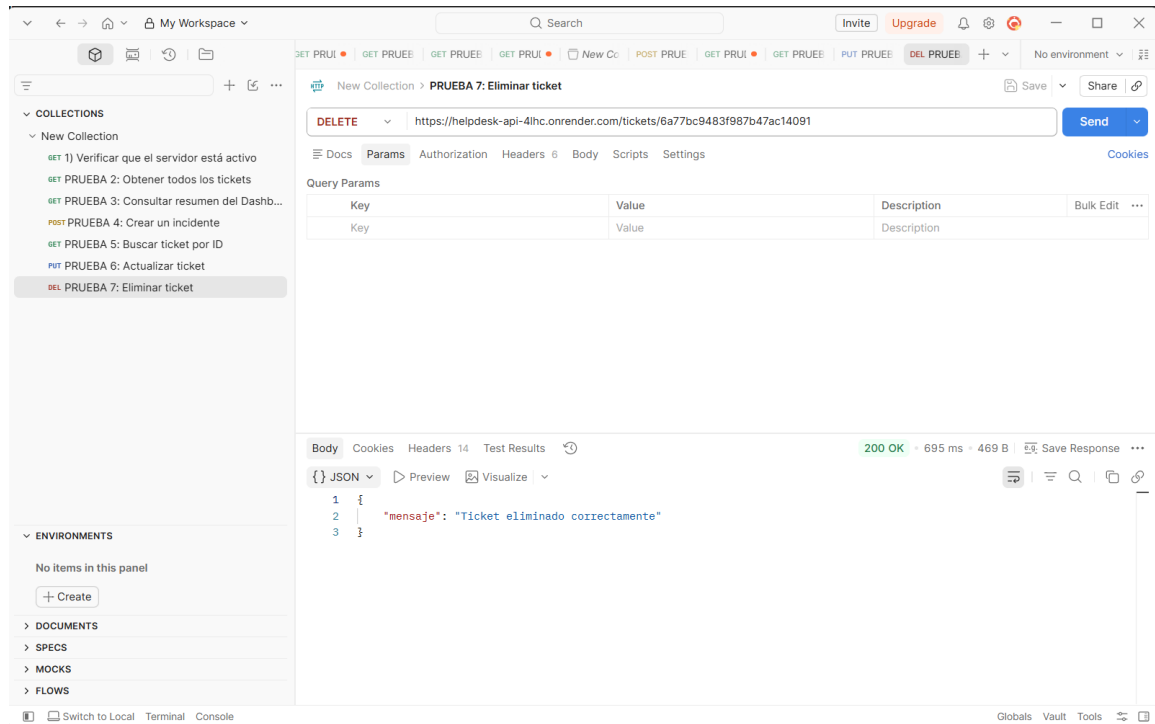


Nota. Mostrar PUT <https://helpdesk-backend-ymmj.onrender.com/tickets/6a76871a53980998f302be38> con los campos modificados.



Figura 10

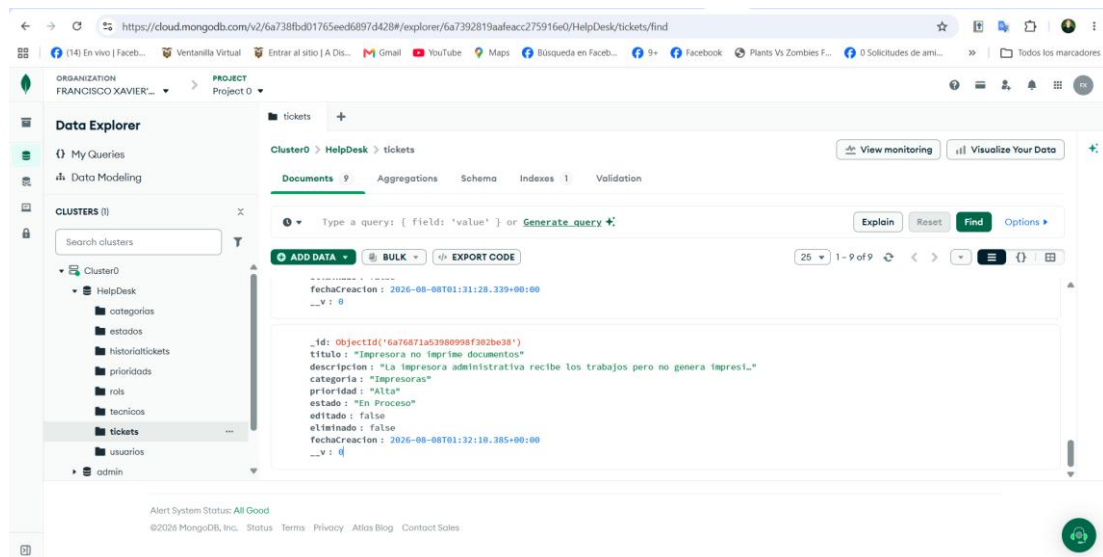
PRUEBA 7: Eliminar ticket



Nota. Mostrar DELETE <https://helpdesk-api-4lhc.onrender.com/tickets/6a77bc9483f987b47ac14091> y el mensaje de eliminación.

Figura 11

Persistencia remota en MongoDB Atlas

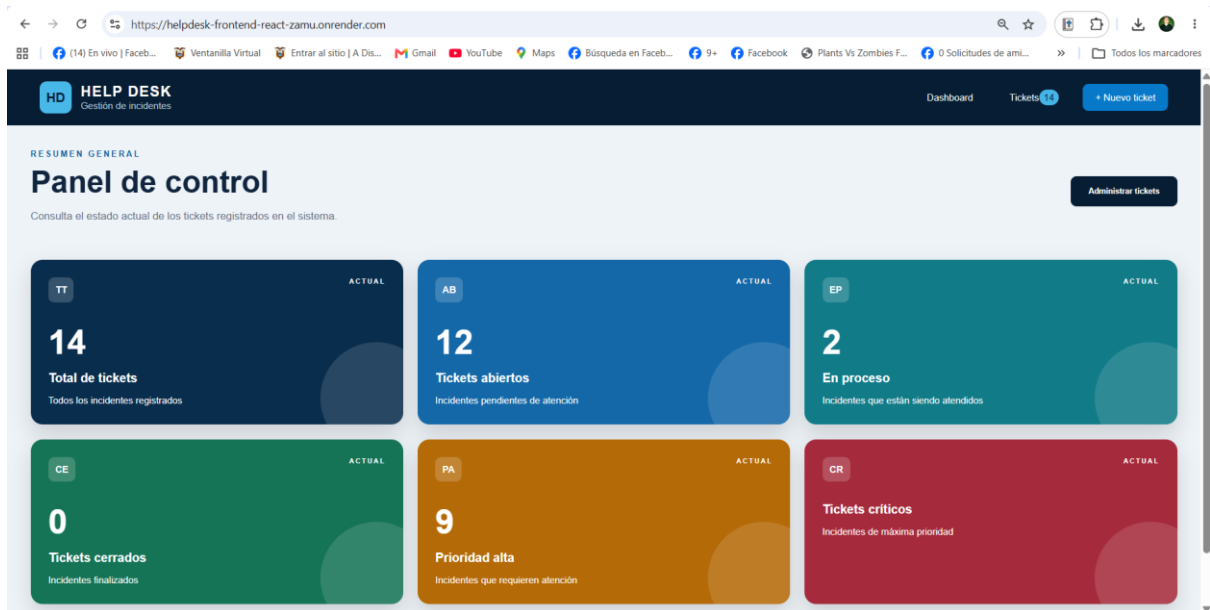


Nota. Data Explorer debe mostrar Cluster0 > HelpDesk y la colección tickets; no exponer credenciales.



Figura 12

Dashboard del frontend



Nota. Mostrar las seis tarjetas con colores únicos.

Figura 13

Formulario de registro o edición

The screenshot shows the 'NUEVO INCIDENTE' section with the title 'Registrar ticket'. Below the title is a form with the following fields:

- Título del incidente:
- Descripción:
- Categoría:
- Prioridad:
- Estado:
- Usuario:

At the bottom of the form are two buttons: 'Guardar ticket' and 'Cancelar'.

Nota. Mostrar campos y botones.



Figura 14

The screenshot shows the 'Editar ticket' form in the HELP DESK application. The form is titled 'NUEVO INCIDENTE' and 'Editar ticket'. It contains the following fields:

- Título del incidente: Prueba API Help Desk
- Descripción: Creación de ticket desde Postman
- Categoría: Software
- Prioridad: Alta
- Estado: Abierto
- Usuario: Administrador

At the bottom of the form are two buttons: 'Guardar ticket' and 'Cancelar'.

The screenshot shows the ticket list view in the HELP DESK application. A modal dialog is displayed over the list, asking '¿Desea eliminar este ticket?' with 'Aceptar' and 'Cancelar' buttons. The list contains three tickets:

Ticket Title	Status
Prueba API Help Desk	Abierto
Problema con acceso al servidor	Abierto
Necesito permiso para acceder a la carpeta compartida del proyecto X	Abierto

Each ticket card shows details like description, category, priority, and user, along with 'Editar' and 'Eliminar' buttons.

Nota. Mostrar listado, filtros y acciones.



Figura 15

Repositorios GitHub de backend y frontend

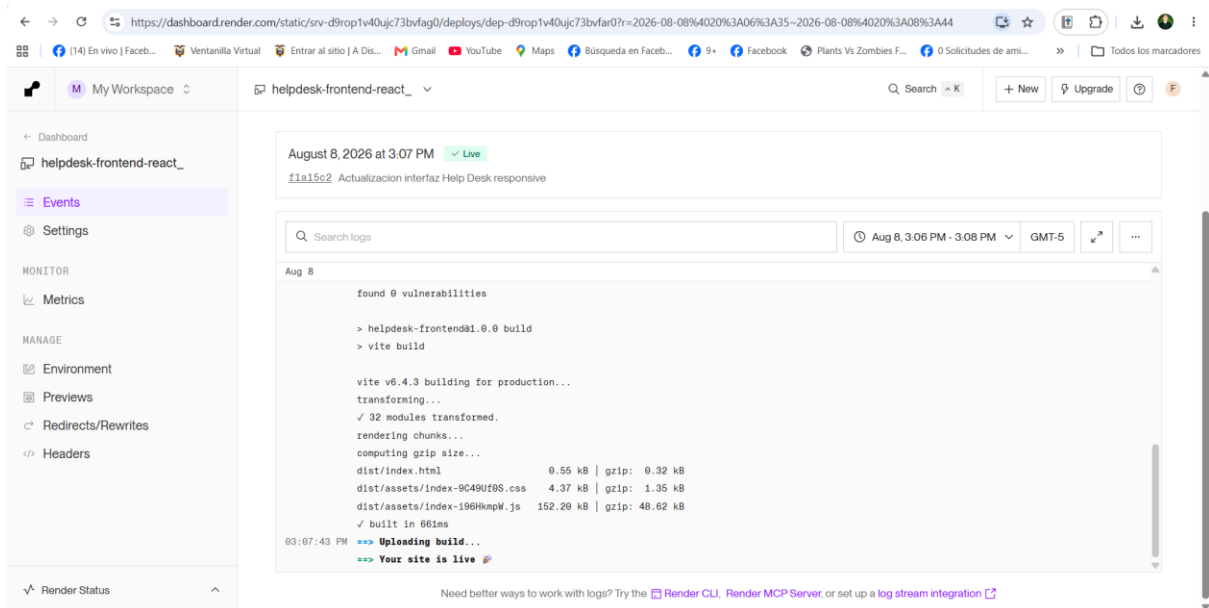
The image shows two screenshots of GitHub repositories. The top screenshot is for the 'HelpDesk-Backend' repository by user 'freyes7194-svg'. It shows the 'develop' branch with 2 branches and 0 tags. The repository has 11 commits and a recent push 30 minutes ago. The file list includes 'config', 'models', 'src', '.gitignore', 'README.md', 'Reyes_Francisco_Actividad#8_.pdf', and 'narkane-lock.json'. The bottom screenshot is for the 'helpdesk-frontend-react' repository by user 'freyes7194-svg'. It shows the 'develop' branch with 3 branches and 0 tags. The repository has 9 commits and a recent push 6 minutes ago. The file list includes 'public', 'src', '.env', '.gitignore', and 'index.html'. Both repositories have a 'Compare & pull request' button and a 'Contribute' button.

Nota. Mostrar HelpDesk-Backend en develop y helpdesk-frontend-react en main; .env y credenciales no deben aparecer.



Figura 16

Selección de Static Site para publicar el frontend en Render



Nota. Esta evidencia muestra que el cliente React + Vite se configuró como sitio estático y no como Web Service de Ruby.

Enlaces de evidencias

Repositorio backend: <https://github.com/freyes7194-svg/HelpDesk-Backend.git>

Repositorio helpdesk-frontend-react: <https://github.com/freyes7194-svg/helpdesk-frontend-react.git>

HelpDesk-Backend: <https://helpdesk-api-4lhc.onrender.com>

helpdesk-frontend-react: <https://helpdesk-frontend-react-zamu.onrender.com>

Google Drive video:
<https://drive.google.com/drive/folders/1cDgLtPoqtLg4lC5AjdeBZbZGWd6k-qIg?usp=sharing>